Introduction

Motivation I

Modern research software repositories increasingly adopt monorepo designs in which multiple projects share a common set of curated resources. A monorepo consolidates source code, documentation, datasets, and governance artefacts under one version-controlled root, enabling atomic cross-project changes and a single source of truth for shared data (Fowler 2002). The practical benefit is significant: a bibliography updated once in `fonds/templates/template_bibliography/` is immediately available to every project that discovers it at runtime, without any per-project copy.

Motivation I

Three categories of shared resource appear consistently across research template repositories:

Motivation I

1. **Data pools (fonds)**: curated reference sets — bibliographies, contact registries, dataset catalogues — that projects query but must never mutate.
2. **Governance rules**: machine-readable constraint schemas and human-readable style guidelines that projects load to validate their own outputs.
3. **Executable tools**: script-based entry points that projects invoke to run computations, validate artefacts, or call external agents.

Motivation I

Without a canonical integration pattern for consuming these resources, projects face a dilemma: they can hard-code discovery paths (creating fragile, repo-root-sensitive logic) or skip resource consumption entirely (forfeiting the monorepo's collaborative benefits). Neither outcome is acceptable in a public, forkable template repository intended to demonstrate best practices (Wilson et al. 2014).

This paper introduces `template_pools_rules_tools`, a **meta-project exemplar** that resolves this dilemma with a four-module architecture (fig. 1). Each module handles one resource category plus a fourth orchestration module:

Contribution I

Module	Resource category	Key function
`src/template_pools_rules_tools/tools/fonds_reader.py`	Data pools	`read_all_fonds()`
`src/template_pools_rules_tools/rules/rules_applier.py`	Governance rules	`validate_against_rules()`
`src/template_pools_rules_tools/tools/tools_invoker.py`	Executable tools	`discover_tools()`
`src/template_pools_rules_tools/tools/integration.py`	All three	`run_integration_demo()`

Contribution I

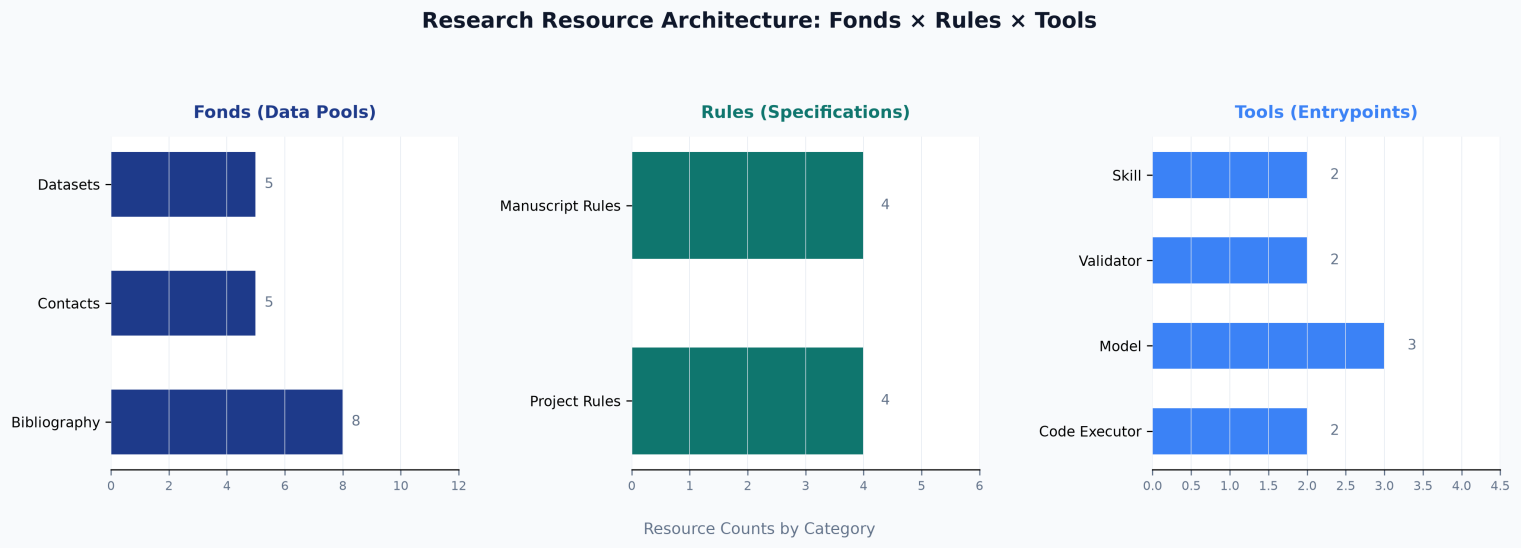


Figure 1: Three-layer resource architecture of template_pools_rules_tools. Fonds (left) provide read-only data pools; Rules (centre) provide governance constraints; Tools (right) provide executable entry points. The Integration layer (bottom) orchestrates all three.

Contribution I

The architecture obeys three design invariants:

Contribution I

- **Read-only resource access**: no module writes to `fonds/`, `rules/`, or `tools/`. The Layer Contract in `AGENTS.md` enforces this at code-review time.
- **Repo-root-relative discovery**: all path resolution uses `pathlib.Path(__file__).resolve().parents[N]` so that scripts work from any working directory.
- **Graceful degradation**: every reader checks file existence before parsing and catches `yaml.YAMLError` around the parse itself, logging a warning and returning a safe empty value either way. The pipeline never raises on a missing or malformed resource.

Three broad alternatives to the fonds/rules/tools pattern are common in research-software monorepos, and each has a known failure mode this design deliberately avoids.

Shared library packages. A monorepo could publish its bibliography, contacts, and datasets as an installable Python package (`pip install monorepo-shared-data`) rather than a filesystem convention. This works well for stable, versioned data but reintroduces a packaging and release cycle for data that changes far more often than code — every bibliography update would require a version bump and a re-install across every consuming project, which is precisely the coordination overhead a monorepo is meant to eliminate.

Symlink-based sharing. Projects could symlink directly into a shared resource directory rather than discovering it at runtime through a reader module. This avoids the reader module's implementation cost but loses the schema-validation and graceful-degradation layers: a symlinked file that is malformed YAML fails wherever it is read, with no structured `status` the pipeline can act on, and no compatibility check against the manifest's `version` field.

Environment-variable configuration. Resource locations could be injected via environment variables (`FONDS_ROOT`, `RULES_ROOT`) rather than resolved relative to the repository root. This is the standard twelve-factor-app pattern for services, but it is a poor fit for a forkable template repository: a contributor who clones the repository and runs a script expects it to work without any environment setup, and environment variables are precisely the kind of implicit, undocumented dependency that a public exemplar should not require.

Related Work and Alternative Designs I

The manifest-and-reader pattern adopted here — versioned typed manifests, repo-root-relative discovery, and graceful degradation — sits between these alternatives: no packaging overhead, structured validation, and zero required environment configuration. fig. 6 shows how this pattern integrates into the project's script pipeline end-to-end.

The remainder of this paper is structured as follows. sec. 4 describes the fond layer and the `fonds_reader` module, including the fond schema taxonomy (fig. 2). sec. 5 describes the rules layer and the `rules_applier` module, including the soft/strong rule hierarchy (fig. 3). sec. 6 describes the tool layer and the `tools_invoker` module, including the tool invocation contract (fig. 4). sec. 7 presents the unified integration pipeline, the manuscript variable token system, the three-level resilience design (fig. 8), and the script pipeline (fig. 6). sec. 8 summarises key findings, limitations, and future directions.

Paper Organisation I

The architecture overview in fig. 1 provides a visual map of these relationships. Runtime statistics collected during integration are visualised in fig. 5, and the corresponding per-component pass/partial/missing states are shown in fig. 7.